

Python Journal Template

Directions: Follow the directions for each part of the journal template. Include in your response all the elements listed under the Requirements section. Prompts in the Inspiration section are not required; however, they may help you to fully think through your response.

Remember to review the Touchstone page for entry requirements, examples, and grading specifics.

Name: Learner

Date: 03/15/2024

Final IDE Program Share Link: <https://onlinegdb.com/ABC123>

*****Functional links will not be shared with the learners in the final report sample file and are for representational examples only.**

Complete the following template. Fill out all entries using complete sentences.

PART 1: Defining Your Problem

Task

State the problem you are planning to solve.

Requirements

- Describe the problem you are trying to solve for.
- Describe any input data you expect to use.
- Describe what the program will do to solve the problem.
- Describe any outputs or results the program will provide.

Inspiration

When writing your entry below ask yourself the following questions:

- Why do you want to solve this particular problem?
- What source(s) of data do you believe you will need? Will the user need to supply that data, or will you get it from an external file or another source?
- Will you need to interact with the user throughout the program? Will users continually need to enter data in and see something to continue?
- What are your expected results or what will be the end product? What will you need to tell a user of your program when it is complete?

<Write your journal entry response here>

Casino craps is a dice game in which players bet on the outcome of the roll of a pair of dice. The problem to solve is to help a user better understand the odds of winning and losing at casino craps at an actual casino. I will create a simulation of the craps game to solve this problem. When executed, the program will first provide a sample game as an example for the user. Second, the program will ask the user to input the number of games to play. The program will play the requested number of games. After the last game is played, the program will output (display) the results of the games. The program will store a record of the simulations in an external log file. By playing the game multiple times and reviewing the statistics, users will get a better understanding of their chances of winning.

PART 2: Working Through Specific Examples

Task

Write down clear and specific steps to solve a simple version of your problem you identified in Part 1.

Requirements

Complete the three steps below **for at least two distinct examples/scenarios**.

- State any necessary input data for your simplified problem.
- Write clear and specific steps in English (not Python) detailing what the program will do to solve the problem.
- Describe the specific result of your example/scenario.

Inspiration

When writing your entry below ask yourself the following questions:

- Are there any steps that you don't fully understand? These are places to spend more time working out the details. Consider adding additional smaller steps in these spots.
- Remember that a computer program is very literal. Are there any steps that are unclear? Try giving the steps of your example/scenario to a friend or family member to read through and ask you questions about parts they don't understand. Rewrite these parts as clearly as you can.
- Are there interesting edge cases for your program? Try to start one of your examples/scenarios with input that matches this edge case. How does it change how your program might work?

<Write your journal entry response here>

Scenario 1: Player loses on first roll:

1. Roll two six-sided random dice for the first time, getting a 1 and a 2.
2. Add the values on the top of the two dice, getting 3.
3. The player loses the game.

Scenario 2: Player wins on first roll:

1. Roll two six-sided random dice for the first time, getting a 3 and a 4.
2. Add the values on the top of the two dice, getting 7.
3. The player wins the game.

Scenario 3: Player loses on a few rolls of the dice:

1. Roll two six-sided random dice for the first time, getting a 1 and a 4.
2. Add the values on the top of the two dice, getting 5.
3. The value is not 2, 3, 7, 11, or 12, so the game continues.
4. Roll the two dice again, getting a 3 and a 6.
5. Add the values on the top of the two dice to get 9.

6. That value is not equal to either 7 or 5, so the game continues.
7. Roll the two dice again, getting a 4 and a 3.
8. Add the values on the top of the two dice to get 7.
9. The player loses the game.

PART 3: Generalizing Into Pseudocode

Task

Write out the general sequence your program will use, including all specific examples/scenarios you provided in Part 2.

Requirements

- Write pseudocode for the program in English but refer to Python program elements where they are appropriate. The pseudocode should represent the full functionality of the program, not just a simplified version. Pseudocode is broken down enough that the details of the program are no longer in any paragraph form. One statement per line is ideal.

Help with writing pseudocode

- Here are a few links that can help you write pseudocode with examples. Remember to check out part 3 of the Example Journal Template Submission if you have not already. Note: everyone will write pseudocode differently. There is no right or wrong way to write it other than to make sure you write it clearly and in as much detail as you can so that it should be easy to convert it to code later.
 - <https://www.geeksforgeeks.org/how-to-write-a-pseudo-code/>
 - <https://www.wikihow.com/Write-Pseudocode>

Inspiration

When writing your entry below ask yourself the following questions:

- Do you see common program elements and patterns in your specific examples/scenarios in Part 2, like variables, conditionals, functions, loops, and classes? These should be part of your pseudocode for the general sequence as well.
- Are there places where the steps for your examples/scenarios in Part 2 diverged? These may be places where errors may occur later in the project. Make note of them.
- When you are finished with your pseudocode, does it make sense, even to a person that does not know Python? Aim for the clearest description of the steps, as this will make it easier to convert into program code later.

<Write your pseudocode here>

Function play()

```
Set the rolled value to roll first dice + roll second dice
If the rolled value is equal to 2, 3 or 12
    Set player status to lose
Else If the rolled value is equal to 7 or 11
    Set player status to win
Else
    Set point value to rolled value
    Set the rolled value to roll first dice + roll second dice
    If the rolled value is equal to 7
        Set player status to lose
    Else If rolled value is equal to point value
        Set player status to win
```

```
Else
    Set the rolled value to roll first dice + roll second dice
    If the rolled value is equal to 7
        Set player status to lose
    Else If rolled value is equal to point value
        Set player status to win
    Else
        Reroll again and continue over and over.
```

Main Program

```
Set Wins = 0
Set Losses = 0
Set Total of Win Rolls = 0
Set Total of Loss Rolls = 0
Set Games Played to 0

Input Times to Play from user
Loop until Games Played = Times to Play
    Add 1 to Games Played
    Call function play()
    Get if the player won or lost
    Get the number of rolls
    If the player won
        Add 1 to Wins
        Add the rolls values to the Total of Win Rolls
    Else if player lost
        Add 1 to Losses
        Add the rolls values to the Total of Loss Rolls

Average Number of Rolls Per Win = Total of Win Rolls / Wins
Average Number of Rolls Per Loss = Total of Loss Rolls / Loss
Winning Percentage = Wins / Games Played
Output results
```

PART 4: Testing Your Program

Task

While writing and testing your program code, describe your tests, record any errors, and state your approach to fixing the errors.

Requirements

- For each test case, describe how your choices for the test helped you understand whether the program was running correctly or not.

For each error that occurs while writing and testing your code:

- Record the details of the error from your IDE. A screenshot or copy-and-paste of the text into the journal entry is acceptable.
- Describe what you attempted in order to fix the error. Clearly identify what approach was the one that worked.

Inspiration

When writing your entry below ask yourself the following questions:

- Have you tested edge cases and special cases for the inputs of your program code? Often these unexpected values can cause errors in the operation of your program.
- Have you tested opportunities for user error? If a user is asked to provide an input, what happens when they give the wrong type of input, like a letter instead of a number, or vice versa?

<Record your errors and fixes here>

After changing my initial code in the main program to check for invalid user entries by adding the try and except statements, I noticed it handled the invalid input (no error) but would stop after that and not request another entry from the user. Here was my initial code:

```
def main():
    #play one game
    print("Running one sample game in full:")
    playOneGame()
    number = 0
    #play multiple games based on the entry by the user
    user_input = input("How many games would you want to have tested: ")

    try:
        number = int(user_input)
        playMultipleGames(number)
    except ValueError:
        print("Please enter in a number for the number of games.")
```

And the output did handle the exception of the letter "a" as input but the program stopped. The output looking like this.

```
Running one sample game in full:
(5, 6) total = 11
You win!
How many games would you want to have tested: a
Please enter in a number for the number of games.
>
```

I want the program to prompt the user again to enter another input until a valid number has been entered. I then added a while loop. By using this loop this will repeat while the variable number is equal to zero. The loop will continue to ask the user for a value user until a valid number is set. I changed my code to look like this (while loop statement is bolded):

```
def main():
    #play one game
    print("Running one sample game in full:")
    playOneGame()
    number = 0
    #play multiple games based on the entry by the user
    while number == 0:
        user_input = input("How many games would you want to have tested: ")
        try:
            number = int(user_input)
            playMultipleGames(number)
        except ValueError:
            print("Please enter in a number for the number of games.")
```

Now after adding in the `while` loop, I ran the code again and entered in the letter "a" as input. When doing so, the program then prompted me again to enter in the number of games rather than ending the program. This solves the issue of the program ending now.

```
Running one sample game in full:
(5, 4) total = 9
(2, 1) total = 3
(1, 1) total = 2
(4, 5) total = 9
You win!
How many games would you want to have tested: a
Please enter in a number for the number of games.
How many games would you want to have tested: >
```

PART 5: Commenting Your Program

Task

Submit your full program code, including thorough comments describing what each portion of the program should do when working correctly.

Requirements

- The purpose of the program and each of its parts should be clear to a reader that does not know the Python programming language.

Inspiration

When writing your entry, you are encouraged to consider the following:

- Is each section or sub-section of your code commented to describe what the code is doing?
- Give your code with comments to a friend or family member to review. Add additional comments to spots that confuse them to make it clearer.

<Copy your full program code here, including comments>

Comments are colored in red. There are three modules/classes to this program.

—The main.py file—

```
#This game plays the game of dice called craps where  
#Players would bet on the outcomes of a pair of dice rolls.  
#If the sum of the dice is 2, 3 or 12, the player loses immediately.  
#If the sum of the dice is 7 or 11, they win immediately.  
#The purpose of the program is to simulate the results between two players
```

```
from player import Player
```

```
#Importing the datetime to get the current date and time  
from datetime import datetime
```

```
#This function is created to play a single game and print out the results after each roll.
```

```
def playOneGame():
```

```
    player = Player()  
    while not player.isWinner() and not player.isLoser():  
        player.rollDice()  
        print(player)  
    if player.isWinner():  
        print("You win!")  
    else:  
        print("You lose!")
```

```
#This function is created to play multiple games and outputs the results based on the number of  
games selected.
```

```

def playMultipleGames(number):
    #Initializing the variables
    wins = 0
    losses = 0
    winRolls = 0
    lossRolls = 0

    #Looping through the number of executions
    for count in range(number):
        player = Player()
        hasWon = player.play()
        rolls = player.getNumberOfRolls()
        if hasWon:
            wins += 1
            winRolls += rolls
        else:
            losses += 1
            lossRolls += rolls

    #Calculating the statistics
    print("The total number of wins is", wins)
    print("The total number of losses is", losses)
    if wins > 0:
        print("The average number of rolls per win is %0.2f" % \
              (winRolls / wins))
    if losses > 0:
        print("The average number of rolls per loss is %0.2f" % \
              (lossRolls / losses))

    print("The winning percentage is %0.3f" % (wins / number))
    print("The multi-game has been saved into the log.")

    logStats(wins, losses, winRolls, lossRolls, number)

#This function will log each multi game run to include the date/time and the details of the run
def logStats(wins, losses, winRolls, lossRolls, number):
    file = open("log.txt", "a")

    #Create a date/time object to get the current date and time
    now = datetime.now()
    #Formatting the output of the date and time to dd/mm/YY H:M:S
    dt_string = now.strftime("%d/%m/%Y %H:%M:%S")
    file.write("\n\ndate and time = " + dt_string)
    #Writing the statistics to a file
    file.write("\nThe total number of wins is " + str(wins))
    file.write("\nThe total number of losses is " + str(losses))
    if wins > 0:
        file.write("\nThe average number of rolls per win is " +
                  str(winRolls / wins))
    if losses > 0:
        file.write("\nThe average number of rolls per loss is " +

```

```

        str(lossRolls / losses))
    file.write("\nThe winning percentage is " + str(wins / number))

#The main function as the point of entry
def main():
    #Play one game
    print("Running one sample game in full:")
    playOneGame()
    number = 0
    #Play multiple games based on the entry by the user

    #Loop while a valid number greater than 0 is entered
    while number <= 0:
        user_input = input("How many games would you want to have tested: ")

        #Check if a valid number is entered
        try:
            number = int(user_input)

            #check if the number entered is > 0
            if number <= 0:
                print("Please enter in a positive number.")
                number = 0
            else:
                playMultipleGames(number)
        except ValueError:
            print("Please enter in a number for the number of games.")

if __name__ == "__main__":
    main()

```

—The player.py file—

```

#The player class is to simulate what a player is able to do.

#We will first import the class Die that we have created
from die import Die

#The class Player is the functionality that makes use of the Die class.
class Player(object):

    #The __init__ method will create a pair of dice and initialize the other variables.
    def __init__(self):
        self.firstDie = Die()
        self.secondDie = Die()
        self.roll = ""
        self.rollsCount = 0
        self.startOfGame = True
        self.winner = self.loser = False

```

```

#The getNumberOfRolls will return the rollCount for the number of rolls
def getNumberOfRolls(self):
    return self.rollCount

#The rollDice rolls both of the dice once.
#Then it updates the roll, the won and the lost outcomes.
#Lastly it returns a tuple of the values of the dice.
def rollDice(self):
    #Increment rollCount by 1
    self.rollCount += 1

    #Roll both dice
    self.firstDie.roll()
    self.secondDie.roll()

    #Set the tuple based on the values from each dice
    (v1, v2) = (self.firstDie.getValue(),
                self.secondDie.getValue())

    #Set the roll value to the value of the two dice
    self.roll = str((v1, v2)) + " total = " + str(v1 + v2)

    #The logic of the game is now running, if this is the first game
    if self.startOfGame:
        #Initial value is set to the the value of the two dice
        self.initialSum = v1 + v2
        self.startOfGame = False

        #If the initial sum is equal to 2, 3 or 12, the player is set to have lost
        if self.initialSum in (2, 3, 12):
            self.loser = True
        #If the initial sum is equal to 7 or 11, the player is set to have won
        elif self.initialSum in (7, 11):
            self.winner = True
        #If this is not the first game
    else:
        #We are now checking for the later sum of the values
        laterSum = v1 + v2
        #if it's not the start of the game and a 7 is rolled, the player loses
        if laterSum == 7:
            self.loser = True
        #If the player rolled the same sum as the first sum, the player wins
        elif laterSum == self.initialSum:
            self.winner = True
    return (v1, v2)

#Returns True if the player has won
def isWinner(self):
    """Returns True if player has won."""
    return self.winner

```

#Returns True if the player has lost

```
def isLoser(self):  
    """Returns True if player has lost."""  
    return self.loser
```

#With the game, it's possible that both isWinner and isLower is false

#If this is the case, the game is not finished and we must roll again until one is true.

#Plays a full game and counts the rolls for that game.

#This returns True for a win and False if the player loses

```
def play(self):  
    #We continue to roll as long as both are False  
    while not self.isWinner() and not self.isLoser():  
        self.rollDice()  
    return self.isWinner()
```

#The __str__ method will return the last roll as a string.

```
def __str__(self):  
    """Returns a string representation of the last roll."""  
    return self.roll
```

—The die.py file—

#Importing the randint module

```
from random import randint
```

#The class Die implements a six-sided die

```
class Die:
```

#The __init__ method is used to create a new instance of die with a default value of 1

```
def __init__(self):  
    self.value = 1
```

#The roll method is used to set the value to a random number between 1 and 6

```
def roll(self):  
    self.value = randint(1, 6)
```

#The getValue method returns the top face value of the die

```
def getValue(self):  
    return self.value
```

#The __str__ method returns the string representation of the value of the die

```
def __str__(self):  
    return str(self.getValue())
```

PART 6: Your Completed Program

Task

Provide the IDE share link to your full program code.

Requirements

- The program must work correctly with all the comments included in the program.

Inspiration

- Check before submitting your touchstone that your final version of the program is running successfully.

<Provide the link to your program here>

<https://onlinegdb.com/ABC123>

*****Functional links will not be shared with the learners in the final report sample file and are for representational examples only.**